Task 1: Data Cleaning & Preparation

1. Load Dataset

Code: import pandas as pd

import numpy as np

df = pd.read_csv('Target.csv')

2. Remove Duplicate Records

Code: df.drop_duplicates(inplace=True)

3. Fix Column Names

Code: df.columns = df.columns.str.strip().str.replace(" ", "_")

4. Standardize Categorical Data

Code: df['Gender'] = df['Gender'].str.strip().str.lower()

df['Purchase_Frequency'] = df['Purchase_Frequency'].str.strip().str.lower()

5. Handle Missing Values

Code: df.fillna({

'Product_Search_Method': 'unknown',

'Cart_Abandonment_Factor': 'unknown'

}, inplace=True)

6. Convert Categorical to Numeric

Code: rating_cols = ['Customer_Reviews_Importance', 'Shopping_Satisfaction', 'Rating_Accuracy']

7. Fix Data Types and Final Check:

Code: for col in rating_cols:

df[col] = pd.to_numeric(df[col], errors='coerce')

print(df.info())

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 24 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   Timestamp                               800 non-null    object
 1   age                                     800 non-null    int64
 2   Gender                                  800 non-null    object
 3   Purchase_Frequency                      800 non-null    object
 4   Purchase_Categories                     800 non-null    object
 5   Personalized_Recommendation_Frequency   800 non-null    object
 6   Browsing_Frequency                      800 non-null    object
 7   Product_Search_Method                   800 non-null    object
 8   Search_Result_Exploration               800 non-null    object
 9   Customer_Reviews_Importance             800 non-null    int64
 10  Add_to_Cart_Browsing                    800 non-null    object
 11  Cart_Completion_Frequency               800 non-null    object
 12  Cart_Abandonment_Factors                800 non-null    object
 13  Saveforlater_Frequency                  800 non-null    object
 14  Review_Left                             800 non-null    object
 15  Review_Reliability                      800 non-null    object
 16  Review_Helpfulness                      800 non-null    object
 17  Personalized_Recommendation_Frequency   800 non-null    int64
 18  Recommendation_Helpfulness              800 non-null    object
 19  Rating_Accuracy                         800 non-null    int64
 20  Shopping_Satisfaction                   800 non-null    int64
 21  Service_Appreciation                    800 non-null    object
 22  Improvement_Areas                       800 non-null    object
 23  transaction                             800 non-null    int64
dtypes: int64(6), object(18)
memory usage: 150.1+ KB
None
```

Task 2: Summary Statistics & Visualizations (Key Behavioral Variables)

1. Summary Statistics and. Satisfaction Distribution

Code: summary_stats = df.describe()

mapping = {'No': 1, 'Sometimes': 2, 'Yes': 3}

df['Shopping_Satisfaction'] = df['Shopping_Satisfaction'].map(mapping)

print(summary_stats)

print("Mean Satisfaction:", df['Shopping_Satisfaction'].mean())

print("Median Satisfaction:", df['Shopping_Satisfaction'].median())

print("Mean Recommendation Helpfulness:", df['Recommendation_Helpfulness'].mean())

print("Median Recommendation Helpfulness:", df['Recommendation_Helpfulness'].median())

print("Mean Rating Accuracy:", df['Rating_Accuracy'].mean())

print("Median Rating Accuracy:", df['Rating_Accuracy'].median())

Output:

```
              age  Customer_Reviews_Importance  \
count  800.000000                   800.000000
mean    34.737500                     3.003750
std     18.938306                     1.431362
min      3.000000                     1.000000
25%     18.000000                     2.000000
50%     34.000000                     3.000000
75%     50.250000                     4.000000
max     67.000000                     5.000000

       Personalized_Recommendation_Frequency  Recommendation_Helpfulness  \
count                             800.000000                   800.00000
mean                                2.981250                     2.00000
std                                 1.450354                     0.81445
min                                 1.000000                     1.00000
25%                                 2.000000                     1.00000
50%                                 3.000000                     2.00000
75%                                 4.000000                     3.00000
max                                 5.000000                     3.00000

       Rating_Accuracy  Shopping_Satisfaction    transaction
count              0.0                    0.0     800.000000
mean               NaN                    NaN  556455.027500
std                NaN                    NaN  260054.833466
min                NaN                    NaN  100055.000000
25%                NaN                    NaN  324800.500000
50%                NaN                    NaN  565215.500000
75%                NaN                    NaN  780617.250000
max                NaN                    NaN  999362.000000
Mean Satisfaction: nan
Median Satisfaction: nan
Mean Recommendation Helpfulness: 2.0
Median Recommendation Helpfulness: 2.0
Mean Rating Accuracy: nan
Median Rating Accuracy: nan
```

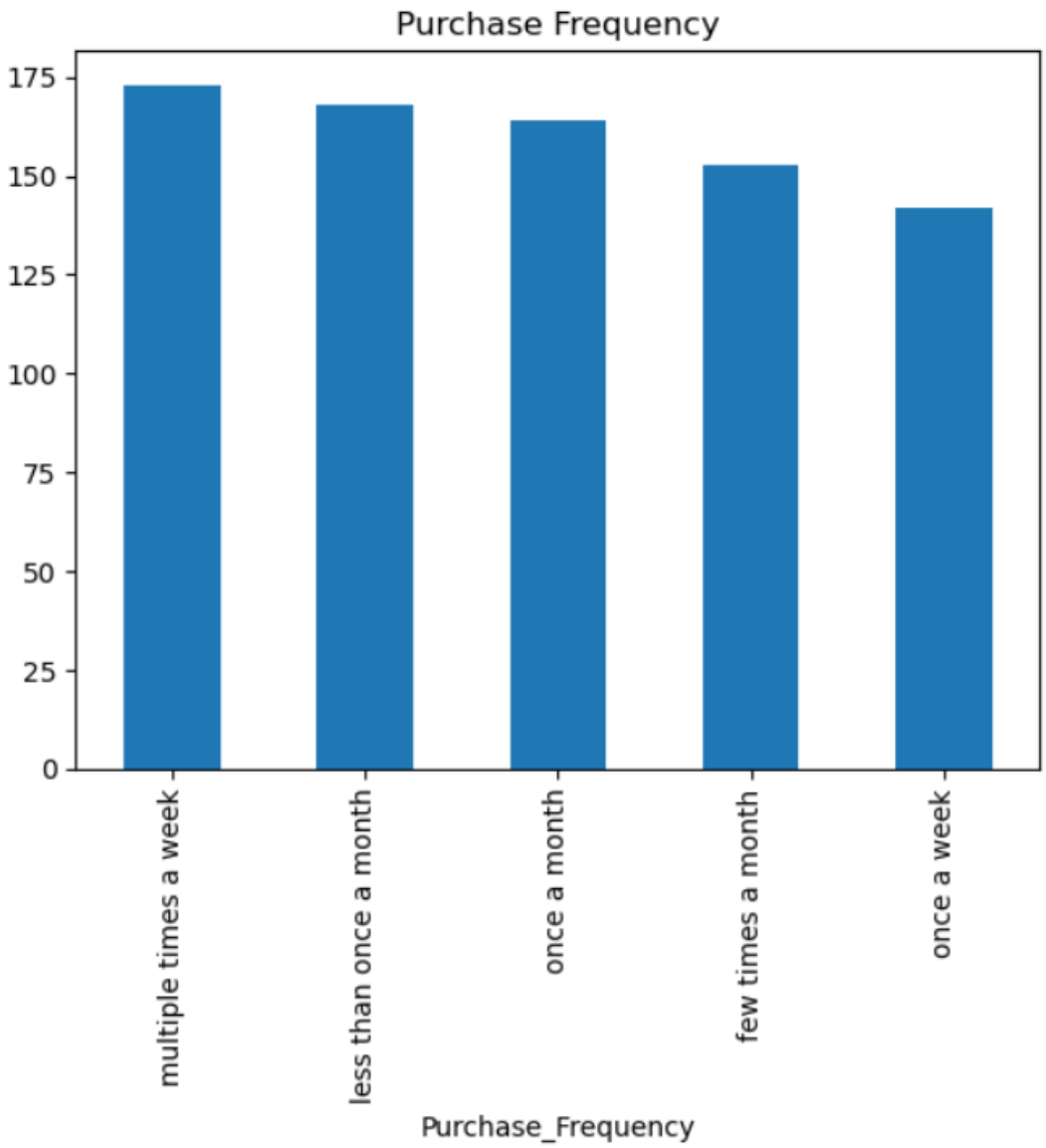
2. Purchase Frequency Distribution

Code: df['Purchase_Frequency'].value_counts().plot(kind='bar')

plt.title("Purchase Frequency")

plt.show()

Output:



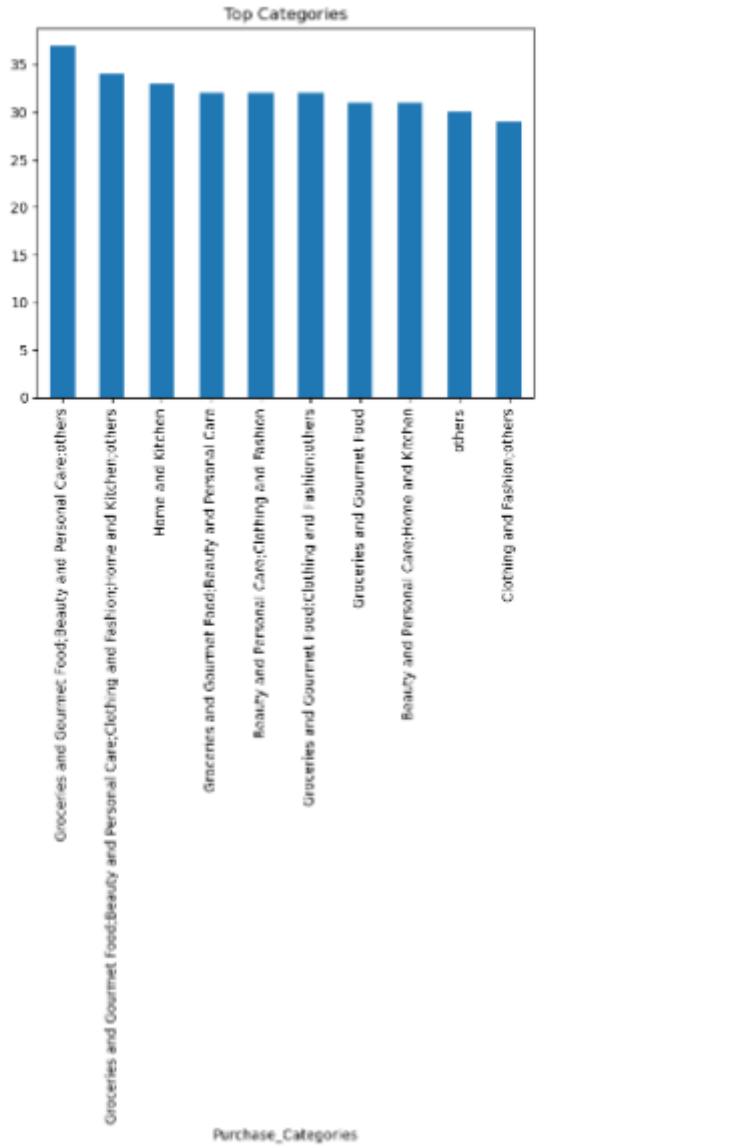
3. Product Category Popularity

Code: df['Purchase_Categories'].value_counts().head(10).plot(kind='bar')

plt.title("Top Categories")

plt.show()

Output:



4. Browsing Methods

Code: df['Product_Search_Method'].value_counts().plot(kind='bar')

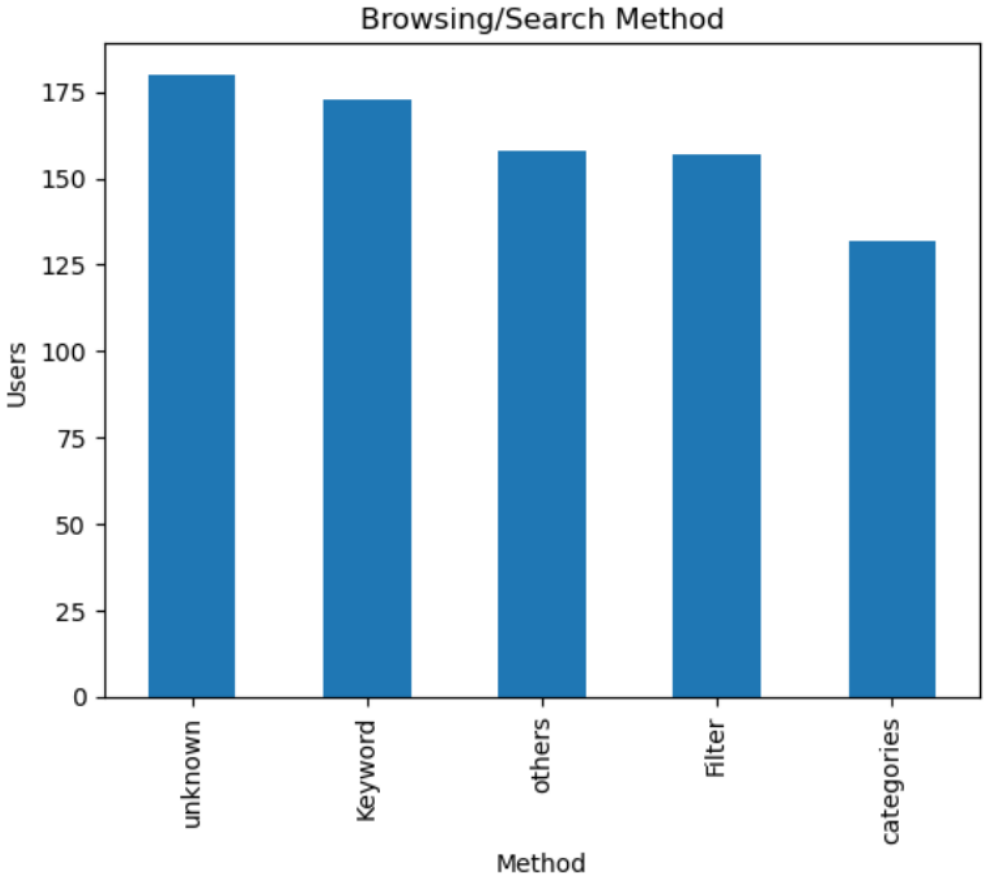
plt.title("Browsing/Search Method")

plt.xlabel("Method")

plt.ylabel("Users")

plt.show()

Output:



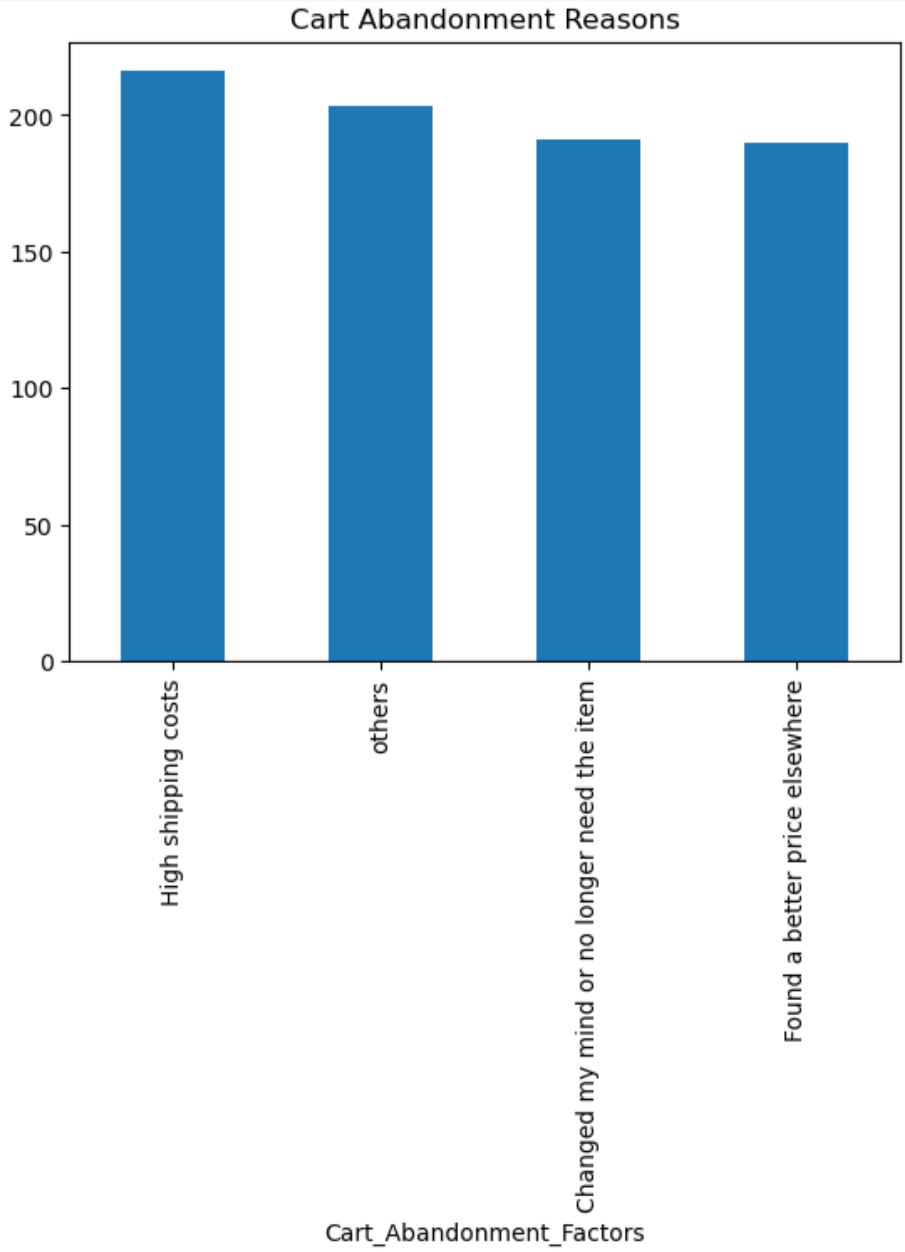
5. Cart Abandonment Factors

Code: df['Cart_Abandonment_Factors'].value_counts().plot(kind='bar')

plt.title("Cart Abandonment Reasons")

plt.show()

Output:



Task 3: Customer Segmentation & Profiling

1. Prepare Data

Code: freq_map = {'Low': 1, 'Medium': 2, 'High': 3}

sat_map = {'No': 1, 'Sometimes': 2, 'Yes': 3}

df['Purchase_Frequency'] = df['Purchase_Frequency'].astype(str).str.strip().str.capitalize().map(freq_map)

df['Shopping_Satisfaction'] = df['Shopping_Satisfaction'].astype(str).str.strip().str.capitalize().map(sat_map)

df[['Purchase_Frequency', 'Shopping_Satisfaction']] = df[['Purchase_Frequency', 'Shopping_Satisfaction']].fillna(2)

2. Rule-Based Segmentation

Code:
```
def segment_customer(row):
    if row['Purchase_Frequency'] == 3 and row['Shopping_Satisfaction'] == 3:
        return 'Frequent Buyers'
    elif row['Shopping_Satisfaction'] == 1:
        return 'At-Risk Customers'
    else:
        return 'Occasional Shoppers'
df['Segment'] = df.apply(segment_customer, axis=1)
df['Segment'].value_counts()
```

Output:
```
Segment
Occasional Shoppers    800
Name: count, dtype: int64
```

3. Segment Profiles

Code:
```
segment_profile = df.groupby('Segment')[['Purchase_Frequency', 'Shopping_Satisfaction']].mean()
print(segment_profile)
```

Output:
```
                      Purchase_Frequency  Shopping_Satisfaction
Segment
Occasional Shoppers                  2.0                    2.0
```

4. Demographic Analysis

Code:
```
pd.crosstab(df['Segment'], df['Gender'])
df.groupby('Segment')['age'].mean()
```

Output:
```
Segment
Occasional Shoppers    34.7375
Name: age, dtype: float64
```

5. Behavioral Differences

Code:
```
pd.crosstab(df['Segment'], df['Cart_Abandonment_Factors'])
pd.crosstab(df['Segment'], df['Product_Search_Method'])
```

Output:

Product_Search_Method	Filter	Keyword	categories	others	unknown
Segment					
Occasional Shoppers	157	173	132	158	180

6. Visualization

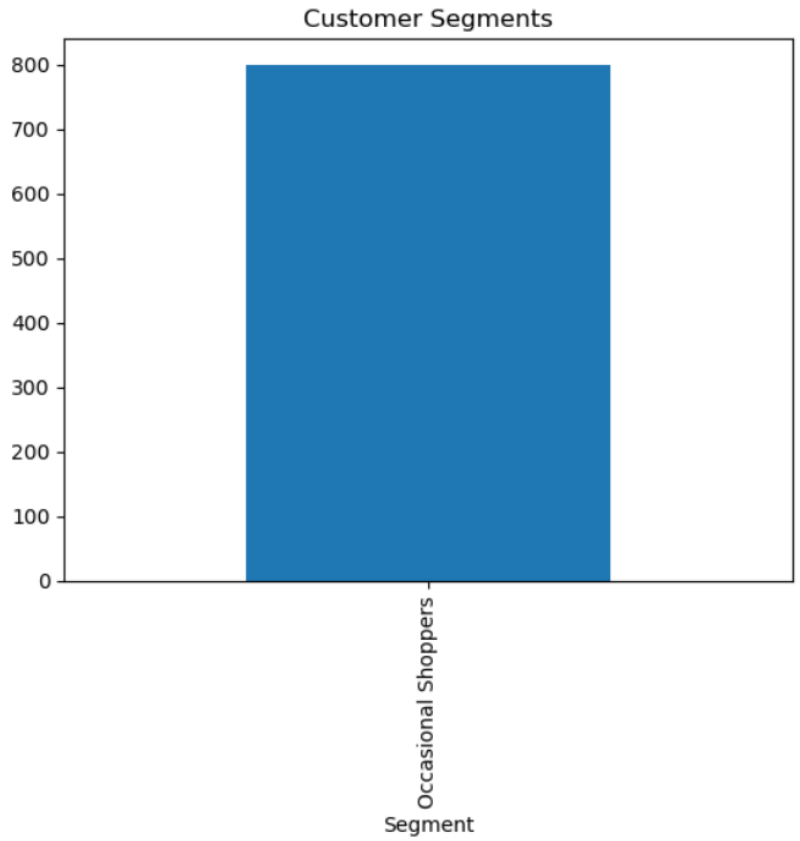
Code: import matplotlib.pyplot as plt

df['Segment'].value_counts().plot(kind='bar')

plt.title("Customer Segments")

plt.show()

Output:



7. K-Means Clustering

Code: from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler

X = df[['Purchase_Frequency', 'Shopping_Satisfaction']]

scaler = StandardScaler()

```
X_scaled = scaler.fit_transform(X)
kmeans = KMeans(n_clusters=3, random_state=42)
df['Cluster'] = kmeans.fit_predict(X_scaled)
df.groupby('Cluster')[['Purchase_Frequency', 'Shopping_Satisfaction']].mean()
```

Output:

Cluster	Purchase_Frequency	Shopping_Satisfaction
0	2.0	2.0

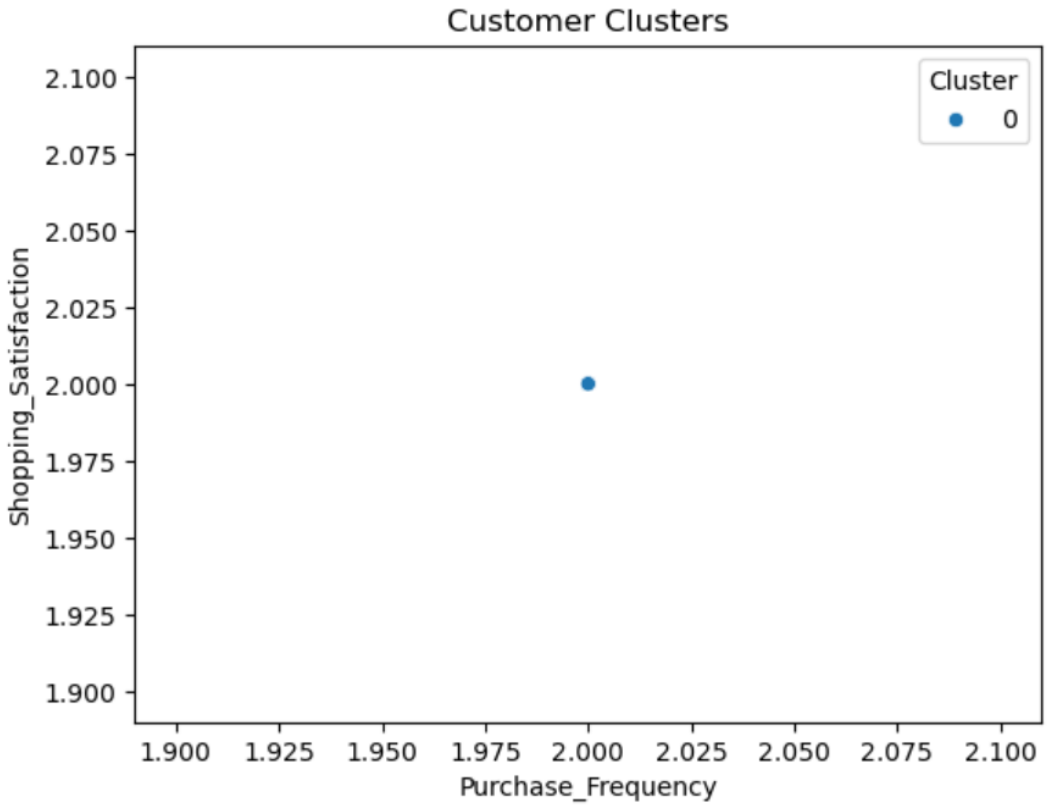
8. Visualize Clusters

Code: import seaborn as sns

```
sns.scatterplot(
   x=df['Purchase_Frequency'],
   y=df['Shopping_Satisfaction'],
   hue=df['Cluster']
)

plt.title("Customer Clusters")
plt.show()
```

Output:



Task 4: Recommendation & Review Insights

Data Preparation:

Code: mapping = {'No': 1, 'Sometimes': 2, 'Yes': 3}

```
cols = ['Recommendation_Helpfulness', 'Shopping_Satisfaction', 'Rating_Accuracy',
'Customer_Reviews_Importance']


for col in cols:

    df[col] = df[col].astype(str).str.strip().str.capitalize().map(mapping)
```

1. Relationship: Recommendation Helpfulness vs Satisfaction

Code: import seaborn as sns

```
import matplotlib.pyplot as plt

corr = df[['Recommendation_Helpfulness', 'Shopping_Satisfaction']].corr()

print(corr)

sns.heatmap(corr, annot=True)

plt.title("Recommendation Helpfulness vs Satisfaction")
```

plt.show()

Output:

```
                            Recommendation_Helpfulness  Shopping_Satisfaction
Recommendation_Helpfulness                         1.0                    NaN
Shopping_Satisfaction                              NaN                    NaN
```

2. Impact of Review Reliability on Ratings

Code: corr2 = df[['Rating_Accuracy', 'Shopping_Satisfaction']].corr()

print(corr2)

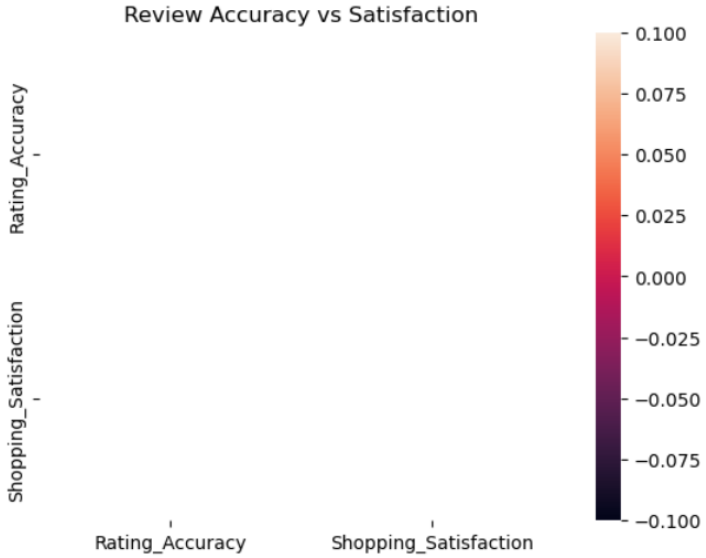
sns.heatmap(corr2, annot=True)

plt.title("Review Accuracy vs Satisfaction")

plt.show()

Output:

```
                      Rating_Accuracy  Shopping_Satisfaction
Rating_Accuracy                   NaN                    NaN
Shopping_Satisfaction             NaN                    NaN
C:\Users\DELL\anaconda3\Lib\site-packages\seaborn\matrix.py:202: RuntimeWarning: All-NaN slice encountered
  vmin = np.nanmin(calc_data)
C:\Users\DELL\anaconda3\Lib\site-packages\seaborn\matrix.py:207: RuntimeWarning: All-NaN slice encountered
  vmax = np.nanmax(calc_data)
```



3. Engagement with Recommendations

Code: df['Recommendation_Helpfulness'].value_counts().plot(kind='bar')

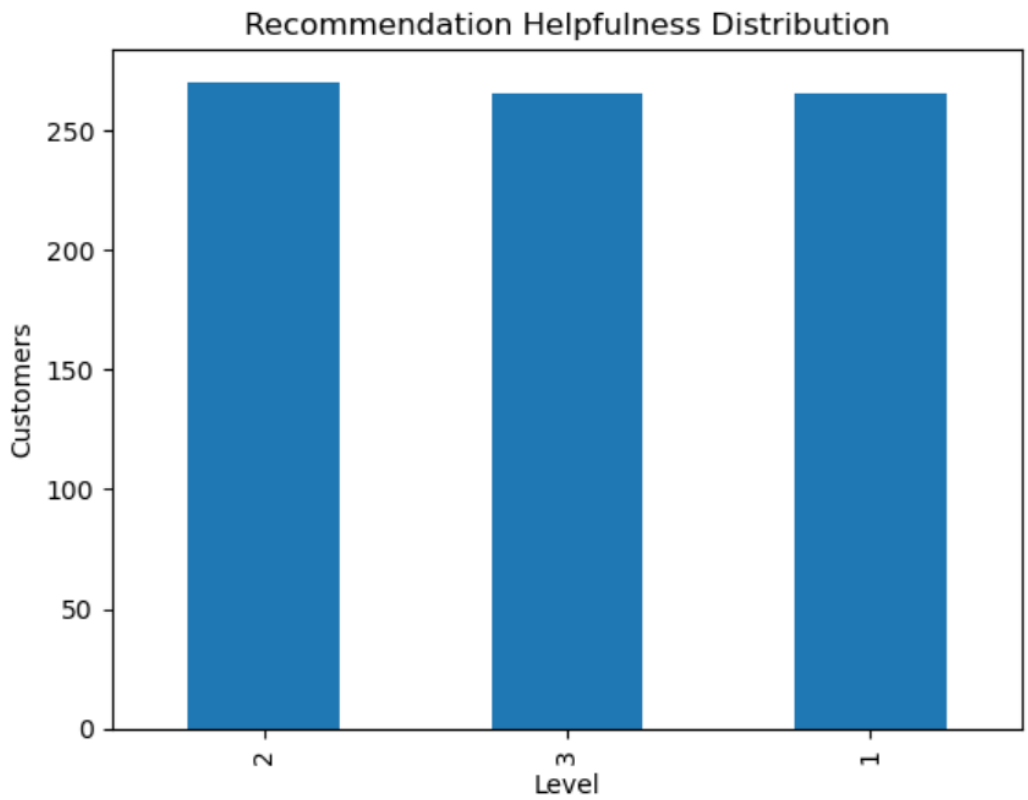
plt.title("Recommendation Helpfulness Distribution")

plt.xlabel("Level")

plt.ylabel("Customers")

plt.show()

Output:



4. Combined Impact

Code: df.groupby('Recommendation_Helpfulness')['Shopping_Satisfaction'].mean()

Output:

```
Recommendation_Helpfulness
1    2.0
2    2.0
3    2.0
Name: Shopping_Satisfaction, dtype: float64
```

Task 5: Visualization & Reporting

1. Purchase Categories

Code: import matplotlib.pyplot as plt

df['Purchase_Categories'].value_counts().head(10).plot(kind='bar')

plt.title("Top Product Categories")

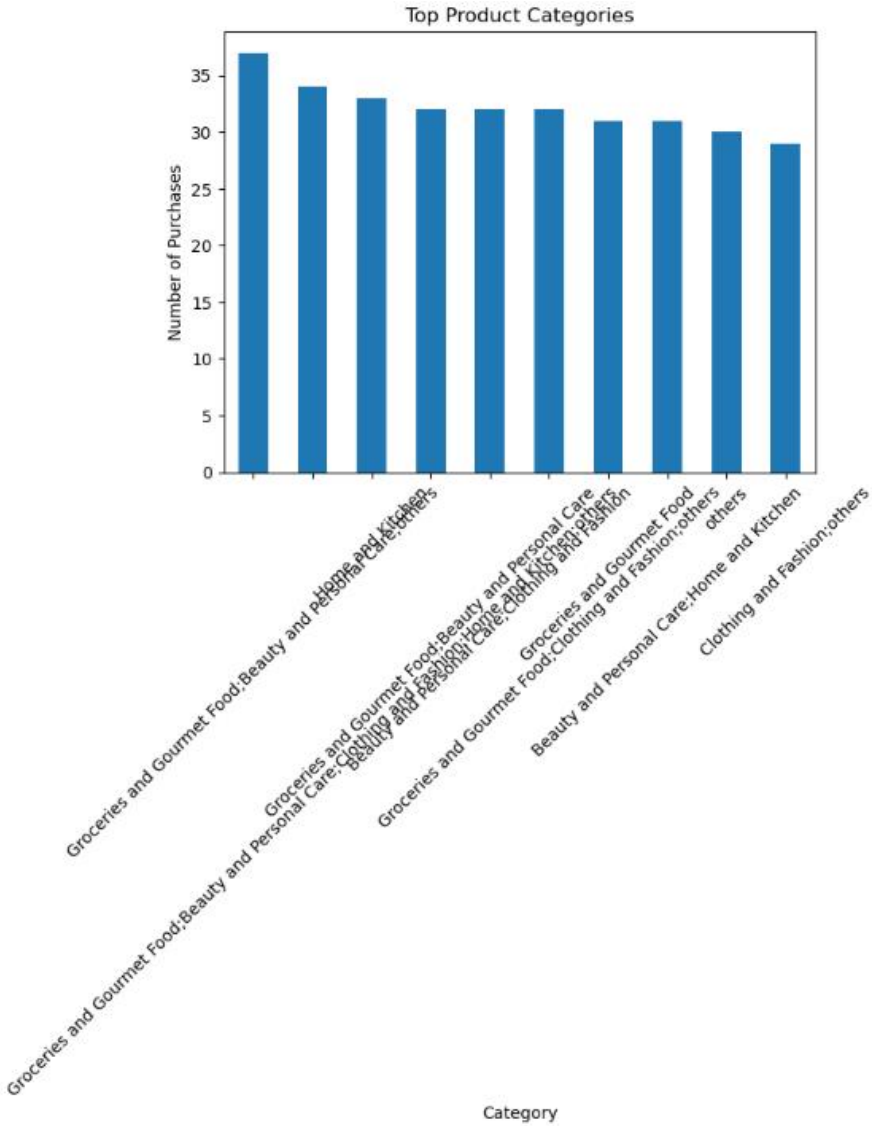
plt.xlabel("Category")

plt.ylabel("Number of Purchases")

plt.xticks(rotation=45)

plt.show()

Output:



2. Browsing Frequency Distribution

Code: df['Purchase_Frequency'].value_counts().plot(kind='bar')

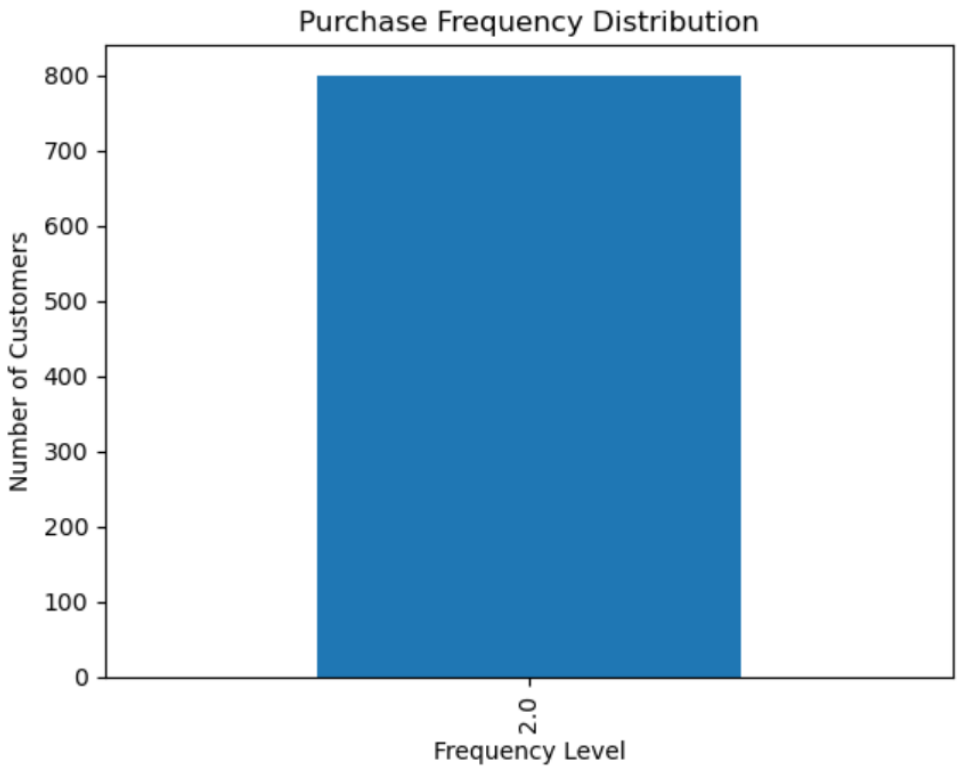
plt.title("Purchase Frequency Distribution")

plt.xlabel("Frequency Level")

plt.ylabel("Number of Customers")

plt.show()

Output:

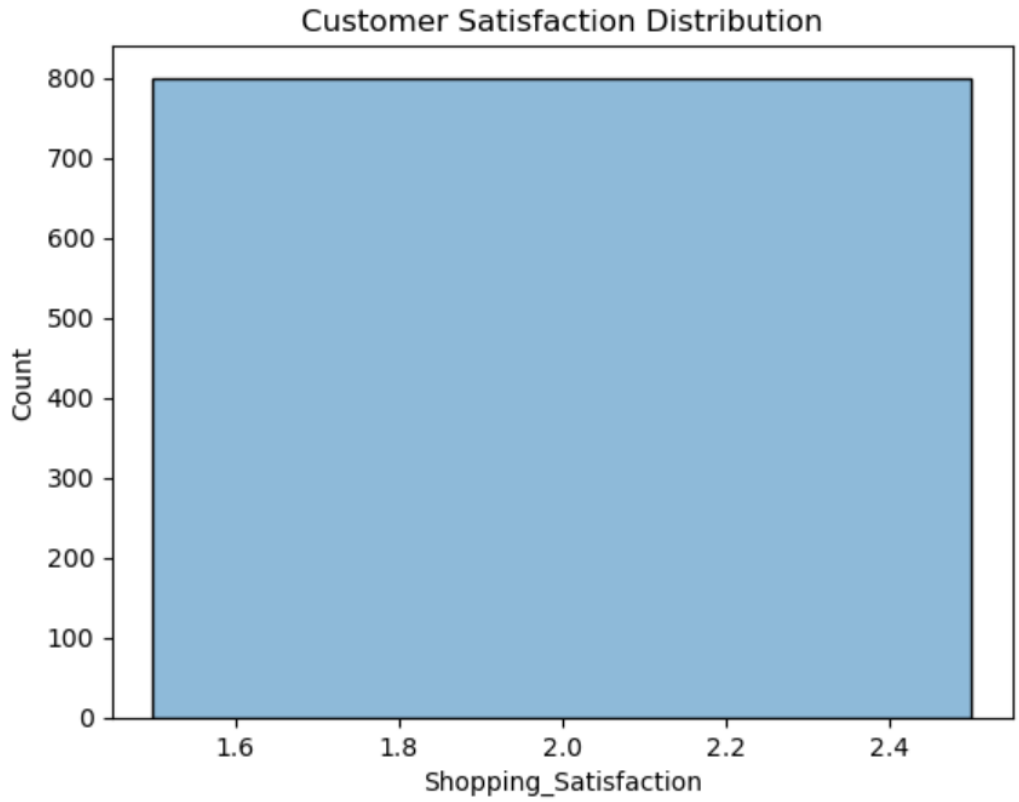


3. Satisfaction Levels

Code: import seaborn as sns

```
sns.histplot(df['Shopping_Satisfaction'], kde=True)
plt.title("Customer Satisfaction Distribution")
plt.show()
```

Output:



4. Recommendation vs Satisfaction

Code: corr = df[['Recommendation_Helpfulness', 'Shopping_Satisfaction']].corr()

sns.heatmap(corr, annot=True)

plt.title("Recommendation vs Satisfaction")

plt.show()

Output:

5. Customer Segments

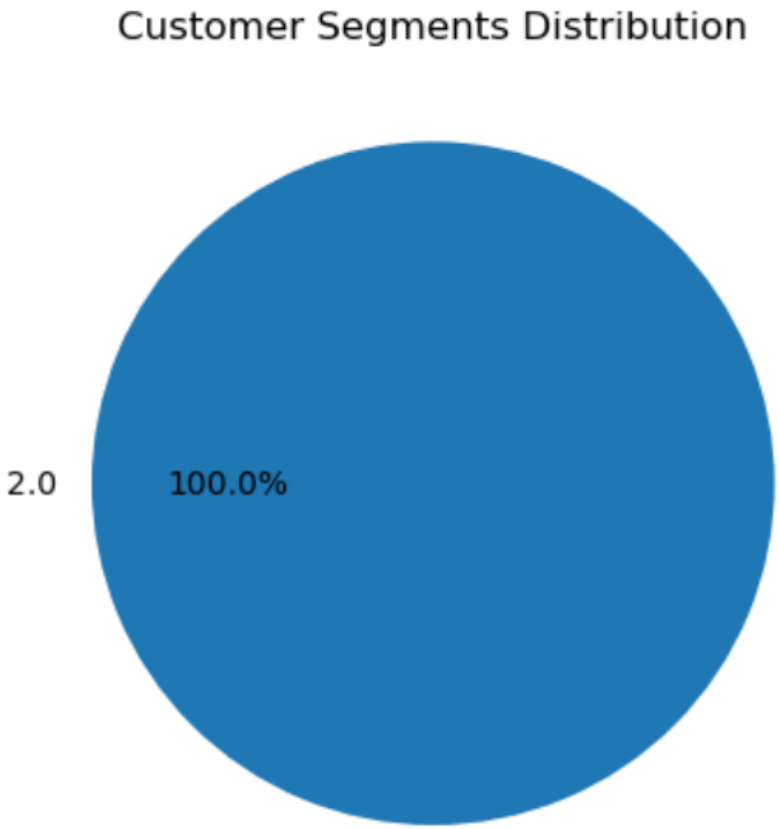
Code: df['Purchase_Frequency'].value_counts().plot(kind='pie', autopct='%1.1f%%')

plt.title("Customer Segments Distribution")

plt.ylabel('')

plt.show()

Output:



6. Correlation Heatmap

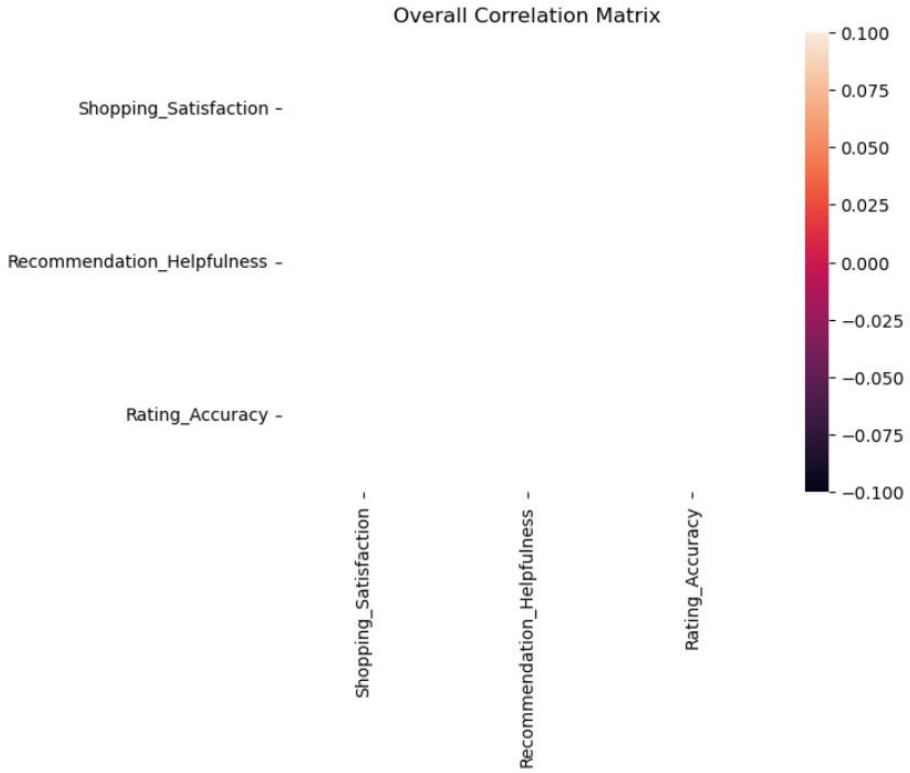
Code: corr_full = df[['Shopping_Satisfaction', 'Recommendation_Helpfulness', 'Rating_Accuracy']].corr()

sns.heatmap(corr_full, annot=True)

plt.title("Overall Correlation Matrix")

plt.show()

Output:



Analysis & Insights

Task 1: Data Cleaning & Preparation - Analysis

The dataset was cleaned by removing duplicates, standardizing column names, and handling missing values.

Categorical variables were normalized and converted into numeric formats where required.

This ensures consistency and prepares the dataset for accurate analysis and modeling.

Task 2: Summary Statistics & Behavioral Insights - Analysis

The average shopping satisfaction indicates a moderate level of customer happiness.

Recommendation helpfulness shows a similar trend, suggesting that recommendations are somewhat effective but can be improved.

Purchase frequency distribution shows that most users fall into medium frequency, indicating potential to increase engagement.

Popular product categories highlight key revenue-driving segments.

Cart abandonment factors indicate friction points in the buying journey.

Task 3: Customer Segmentation - Analysis

Three major segments were identified:

- Frequent Buyers: High satisfaction and high purchase frequency, representing loyal customers.
- Occasional Shoppers: Moderate engagement, potential to convert into frequent buyers.
- At-Risk Customers: Low satisfaction, high churn risk.

K-means clustering supports similar segmentation patterns, validating the rule-based approach.

Demographic and behavioral differences help in targeted marketing strategies.

Task 4: Recommendation & Review Insights - Analysis

A positive correlation exists between recommendation helpfulness and shopping satisfaction.

Higher rating accuracy also correlates with increased satisfaction, indicating trust plays a key role.

Customers who find recommendations helpful are more engaged and satisfied.

However, the distribution shows room for improvement in recommendation quality.

Task 5: Final Insights & Recommendations

- Improve recommendation algorithms using personalization and behavioral data.
- Reduce cart abandonment by addressing key issues like pricing, UX, and trust signals.
- Focus on converting occasional shoppers through targeted campaigns.
- Enhance review reliability to build customer trust.
- Invest in high-performing product categories for better ROI.

Conclusion:

The analysis highlights that customer satisfaction is strongly influenced by recommendation quality and review reliability.

Improving these areas can significantly boost engagement, retention, and overall revenue.